

Software Engineering - 2

Online Food Ordering System

Spring Boot 3.2.5

Java 21

Microservices

JWT Auth

Docker

MongoDB

- **SRS Document:** Software Requirements Specification
- **UML Diagrams:** Use Case · Class · Activity · Sequence
- **ERD:** Unified Entity-Relationship Diagram
- **OCL Constraints:** Formal Object Constraint Language Rules

Table of Contents

- 1. Software Requirements Specification**
- 2. Use Case Diagram**
- 3. Class Diagrams (Microservices)**
- 4. ERD (All System)**
- 5. Activity Diagrams**
- 6. Sequence Diagrams**
- 7. OCL Constraints**
- 8. Team Information**

1. Introduction

1.1 Purpose

The primary purpose of this SRS document is to provide a comprehensive description of the functional and technical requirements for the **Online Food Ordering System**. It serves as a definitive guide for developers, testers, and stakeholders to ensure the system is built according to specified business logic and microservices architecture.

1.2 Scope

The system is a microservices-based application designed to provide a convenient way for customers to purchase food online.

- **Customers:** Can browse restaurants, view menus, apply promo codes, and track orders.
 - **Restaurant Admins:** Enable owners to list items, manage menus, and process orders.
 - **Delivery Riders:** Allow riders to see order details, update delivery status, and view ratings.
 - **Super Admins:** Provide a centralized dashboard to moderate users, manage restaurants, and monitor platform-wide statistics.
-

2. Overall Description

2.1 System Perspective

The system is developed using **Spring Boot** as the backend framework, utilizing **Microservices** and **Spring Cloud** for inter-service communication. Security is centralized at the API Gateway using **JWT (JSON Web Tokens)** for stateless authentication.

2.2 Design and Implementation Constraints

- **Database Isolation:** Data persistence is distributed across isolated databases; no cross-database SQL joins are permitted.
- **Statelessness:** The backend remains stateless, relying on JWTs for session management.
- **Tech Stack:** Standardized on Java 21, Spring Boot 3.2.5, and Docker Compose for containerization.

3. Functional Requirements

- **FR-01:** The system shall allow users to register as a Customer, Owner, or Delivery rider.
- **FR-02:** The system shall authenticate users via email and password, generating a JWT upon success.

- **FR-03:** The system shall support multi-image uploads for menu items.
 - **FR-04:** Customers shall be able to search and filter restaurants by category or discount.
 - **FR-05:** The system shall perform conflict checks for delivery assignments.
 - **FR-06:** Owners shall have the authority to approve, reject, or update order preparation statuses.
-

4. Non-Functional Requirements

4.1 Security

- The system shall use **BCrypt** for password hashing.
- Every user and Admin must change their initial password after the first successful login.

4.2 Performance

- Search queries and filtering results shall be returned in less than 2 seconds.
- Minimum "join" statements are preferred for faster database execution.

4.3 Availability

- The application will be available and working properly 24/7.

- **5. System Diagrams**

- **5.1 Use Case Diagram**

- **The Use Case Diagram for the Online Food Ordering System illustrates the interactions between four primary actors: the Customer, Restaurant Admin, Delivery Rider, and Super Admin. The system employs an inheritance structure where specialized users inherit basic functionalities like Registration and Login from an Abstract User. Strategically, the diagram highlights critical dependencies: placing an order includes confirming payment and validating a JWT at the Gateway to ensure secure, stateless communication.**

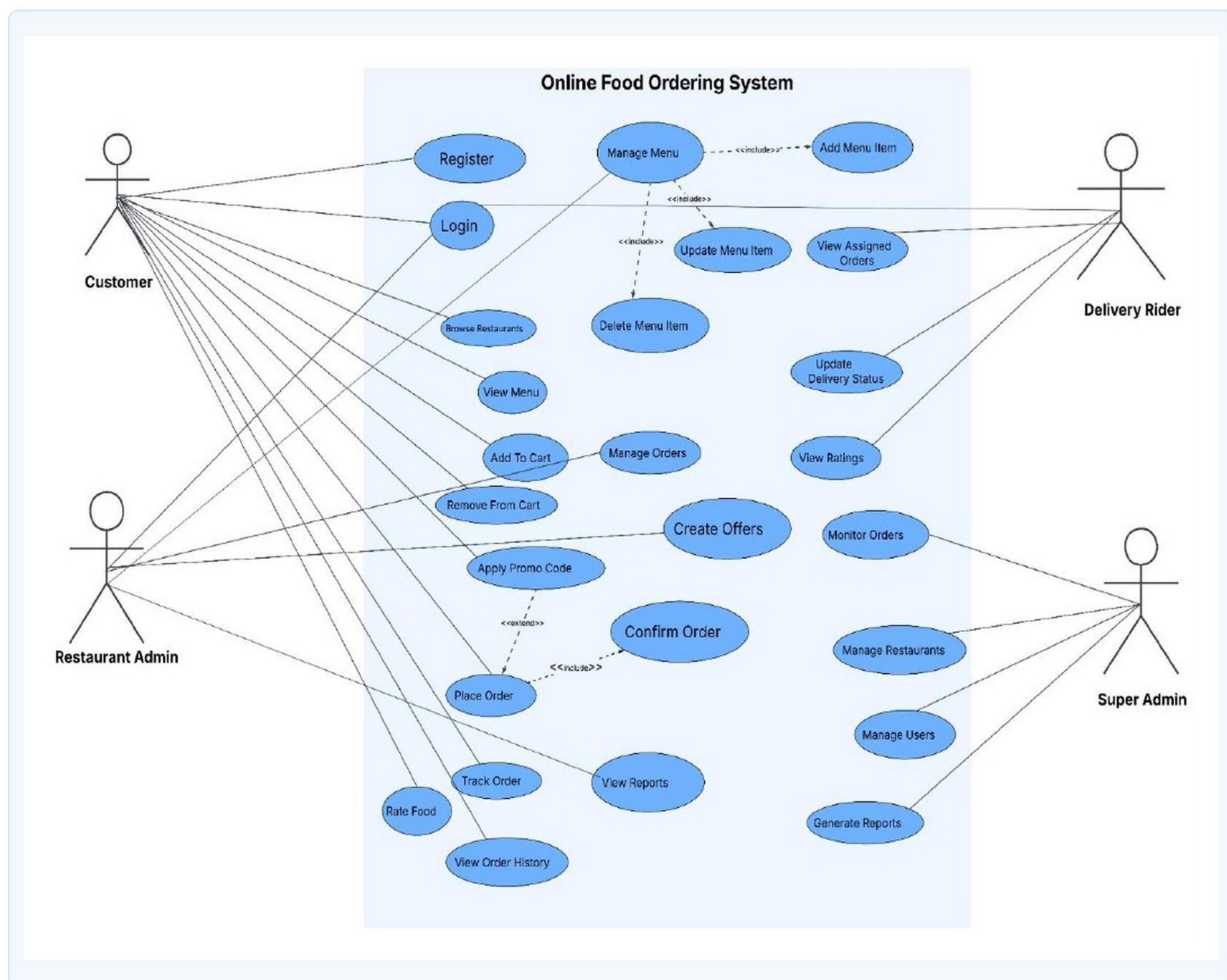
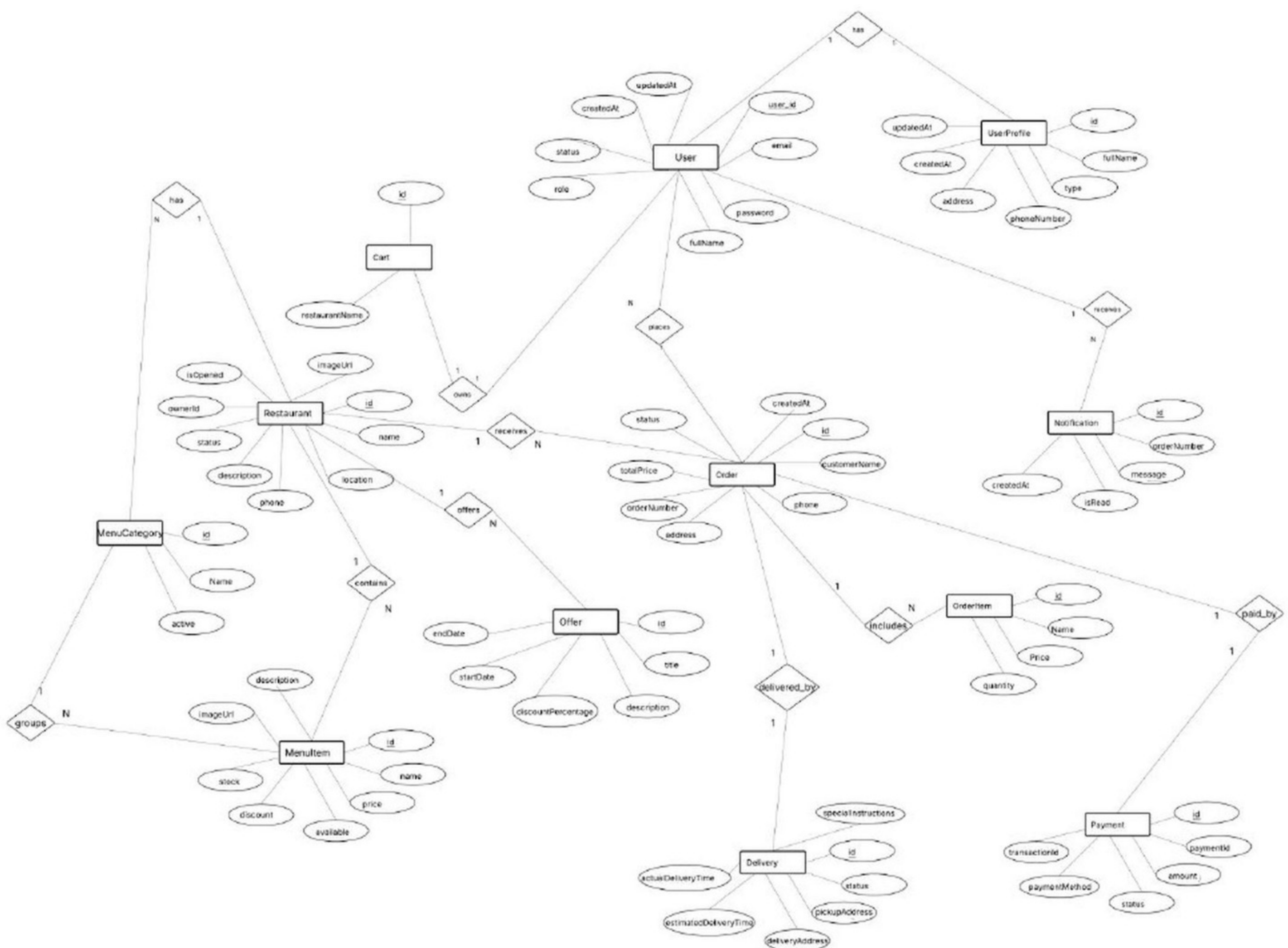


Figure 1 – System Use Case Diagram

5.2 ERD (All System)

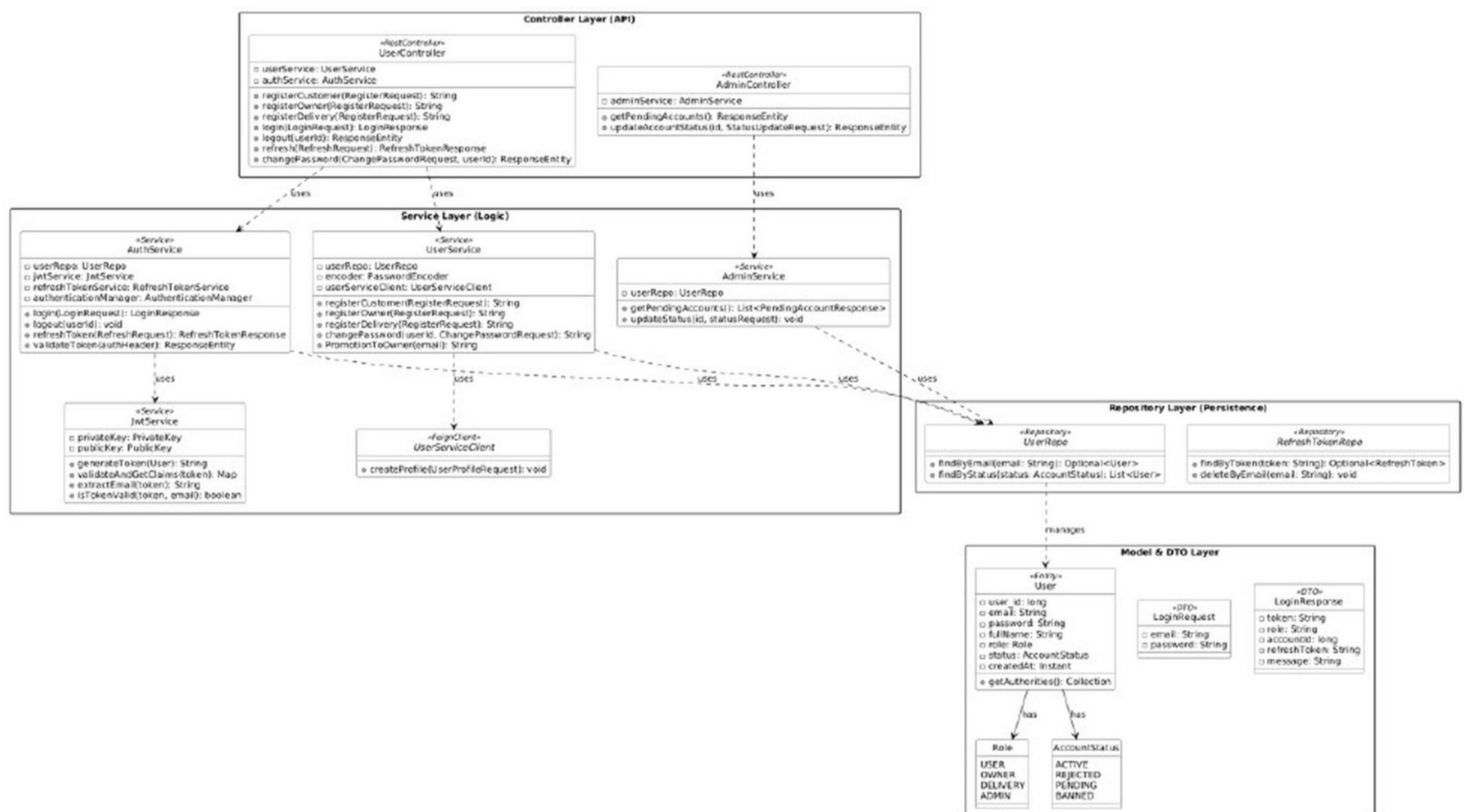
The ERD integrates various microservices into a unified data model. Key entities include:

- **User:** Primary actor linked to profiles and roles.
- **Order:** Captures the intent to purchase, storing a **Monthly Rent/Price Snapshot** for record integrity.
- **Menu Item:** Contains price, stock, and discount attributes.



5.3 Class Diagram (Authentication Service)

The provided Class Diagram illustrates the internal architecture of the Authentication Service, which manages the security layer and handles user registration and JWT generation. It follows a structured Controller-Service-Repository pattern where AuthController and AdminController act as entry points for external API requests. These controllers interact with the AuthService and UserService to manage the User entity, including attributes like email, password hash, and roles.



OCL For Authentication Service class Diagram:1- context

User inv EmailUnique: User.allInstances()->forAll(u | u <> self implies u.email <> self.email)

2- context User inv PasswordMinLength:

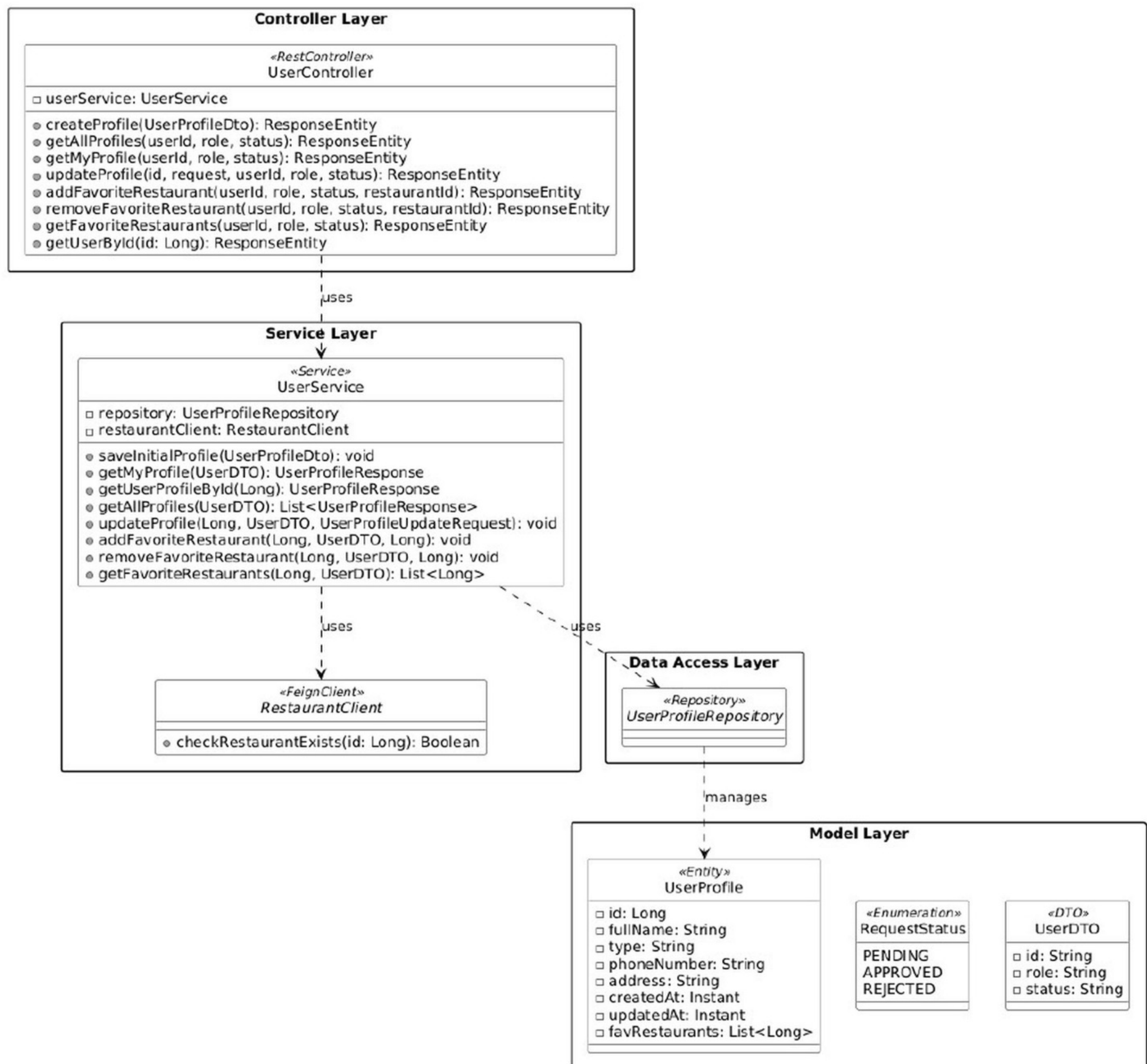
self.password.size() >= 8

3- context User inv AdminMustBeActive: self.role =

Role::ADMIN implies self.status = AccountStatus::ACTIVE

5.4 Class Diagram (User Service)

The provided Class Diagram illustrates the internal architecture of the User Service, which manages detailed user profiles and personalization features like "Favorite Restaurants". It interacts with the Restaurant Service via a FeignClient (RestaurantClient) to verify restaurant data across microservice boundaries.



OCL For User Service class Diagram:

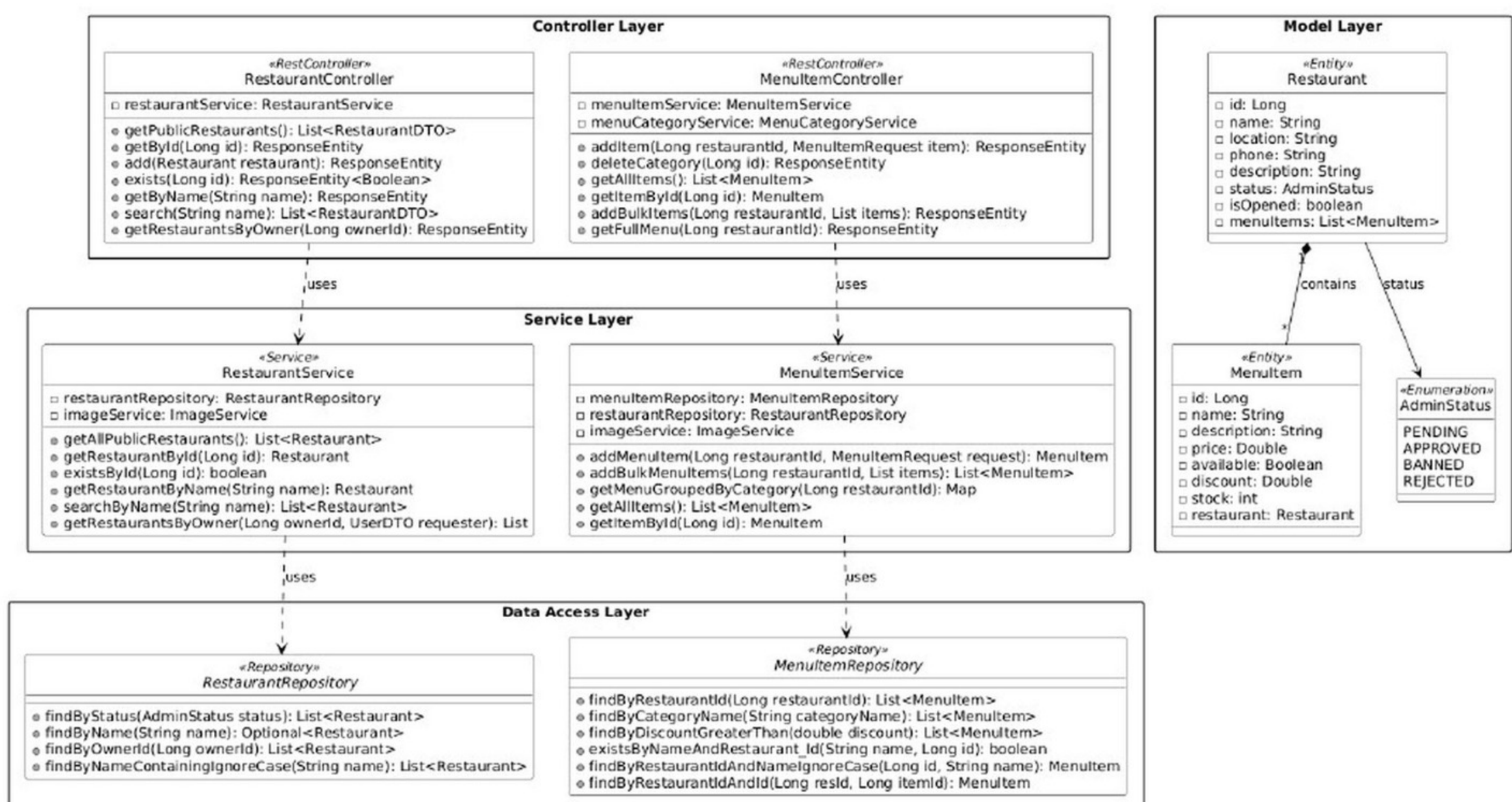
1- context UserProfile inv PhoneValid: self.phoneNumber
<> null and self.phoneNumber.size() >= 7

2- context UserProfile inv TimestampOrder:
self.updatedAt >= self.createdAt

3- context UserService::addFavoriteRestaurant(userId: Long, dto: UserDTO, restaurantId: Long) : void pre RestaurantExists:
 RestaurantClient.checkRestaurantExists(restaurantId) = true

5.5 Class Diagram (Restaurant Service)

The provided Class Diagram illustrates the internal architecture of the Restaurant Service, which handles the lifecycle of restaurants and their menus. The MenuItem entity captures pricing, availability, and stock levels, ensuring customers can only order items currently in stock.



OCL For Restaurant Service class Diagram:

1- context Restaurant inv OpenedRequiresApproved:

`self.isOpened = true implies self.status = AdminStatus::APPROVED`

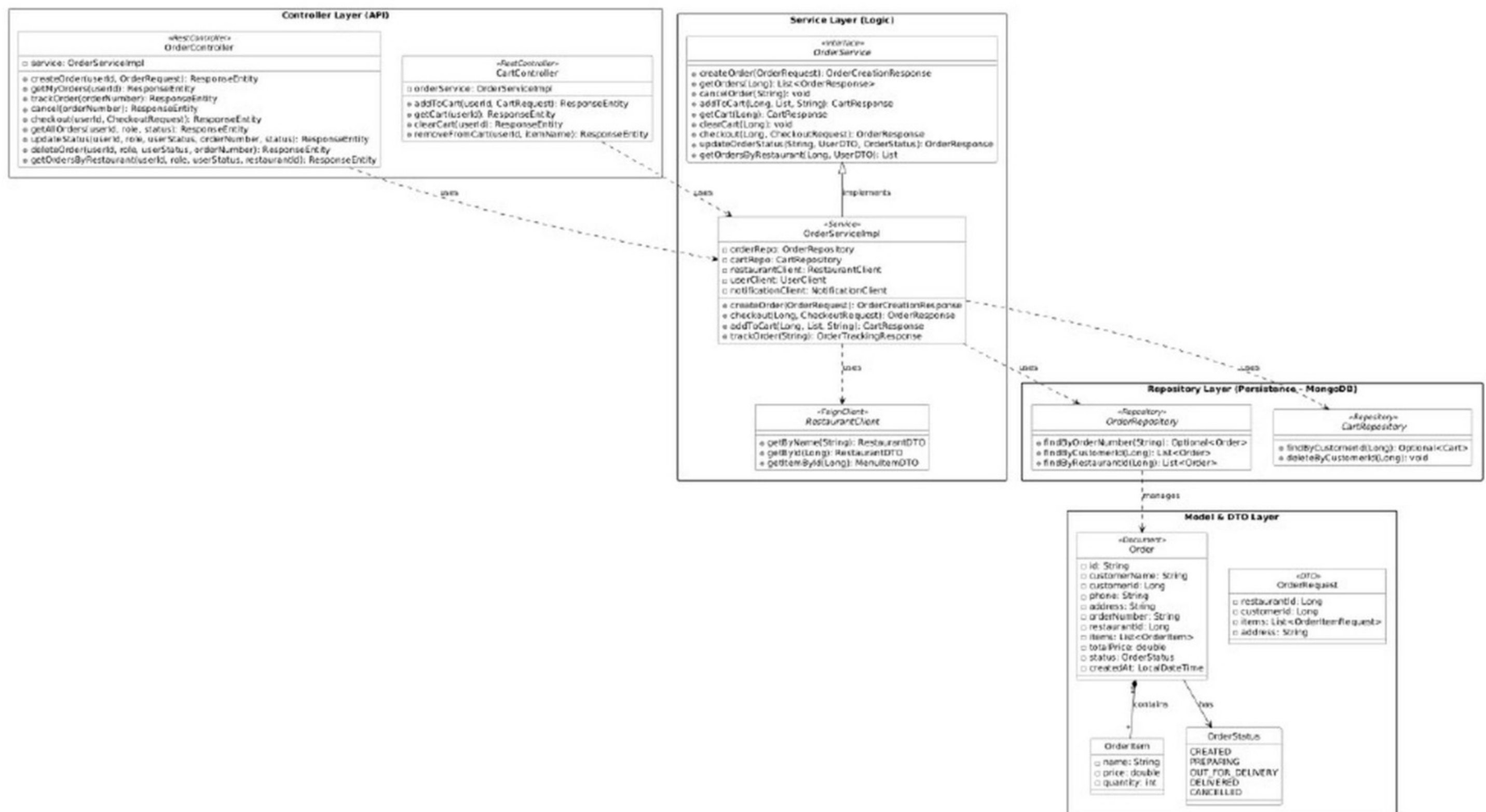
2- context MenuItem inv PricePositive: `self.price > 0`

3- context MenuItem inv UnavailableWhenNoStock:

`self.stock = 0 implies self.available = false`

5.6 Class Diagram (Order Service)

The provided Class Diagram illustrates the internal architecture of the Order Service, which utilizes a MongoDB persistence layer to manage complex order documents. It tracks the order lifecycle from creation through to fulfillment, ensuring financial terms are preserved through price snapshots.



OCL For Order Service class Diagram:

1- context Order inv TotalPriceConsistent: self.totalPrice = self.items->iterate(i; sum: Real = 0.0 | sum + (i.price * i.quantity))

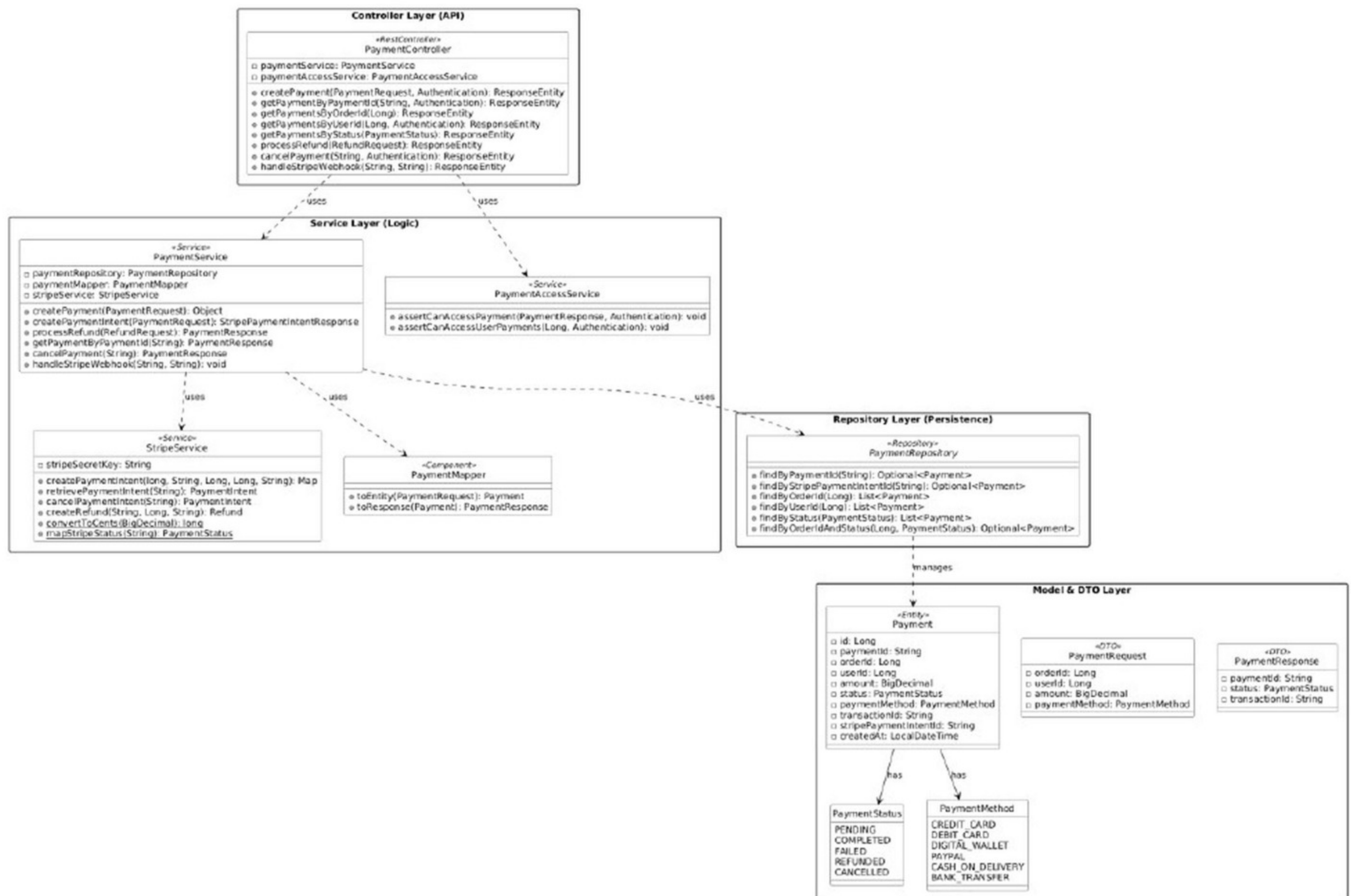
2- context

OrderServiceImpl::updateOrderStatus(orderNumber: String, dto: UserDTO, status: OrderStatus):

OrderResponse pre ValidTransition: let current = Order.allInstances()->select(o | o.orderNumber = orderNumber)->first().status in (current = OrderStatus::CREATED implies status = OrderStatus::PREPARING)

5.7 Class Diagram (Payment Service)

The provided Class Diagram illustrates the internal architecture of the Payment Service, which integrates with the Stripe API to manage transaction statuses. The service ensures financial integrity by enforcing strict rules on completed payments and refund eligibility.



OCL For Payment Service class Diagram:

1- context Payment inv OneCompletedPaymentPerOrder:

Payment.allInstances()->select(p | p.orderId = self.orderId
and p.status = PaymentStatus::COMPLETED)->size() <= 1

2- context Payment inv CashNotRefundable:

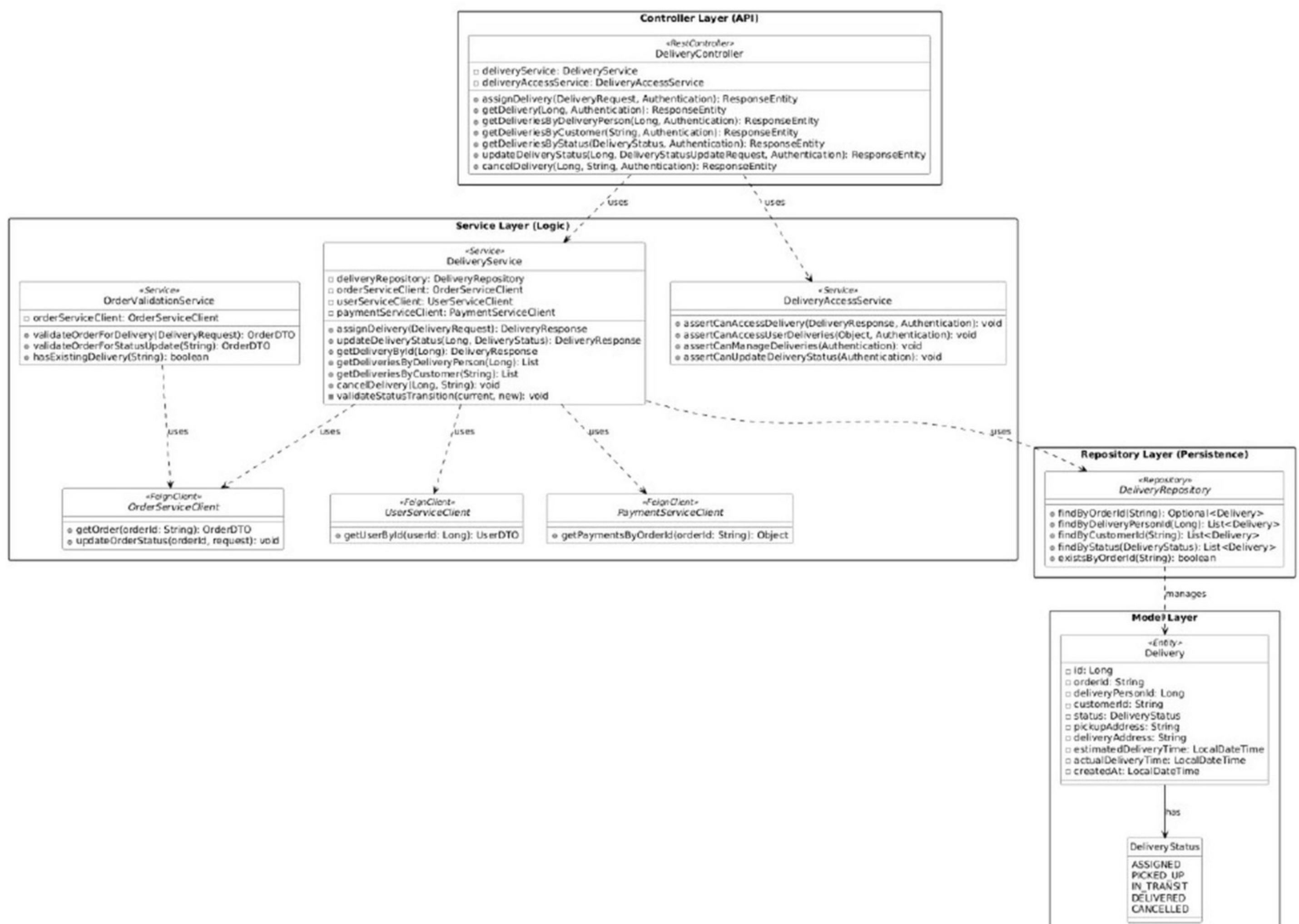
self.paymentMethod =

PaymentMethod::CASH_ON_DELIVERY implies self.status

<> PaymentStatus::REFUNDED

5.8 Class Diagram (Delivery Service)

The provided Class Diagram illustrates the internal architecture of the Delivery Service, which manages the logistics of moving food from restaurants to customers. It coordinates with riders and validates that each rider only handles one active delivery at a time.



OCL For Delivery Service class Diagram:

1- context Delivery inv AddressesDiffer:

`self.pickupAddress <> self.deliveryAddress`

2- context DeliveryService::assignDelivery(request:

DeliveryRequest) : DeliveryResponse pre

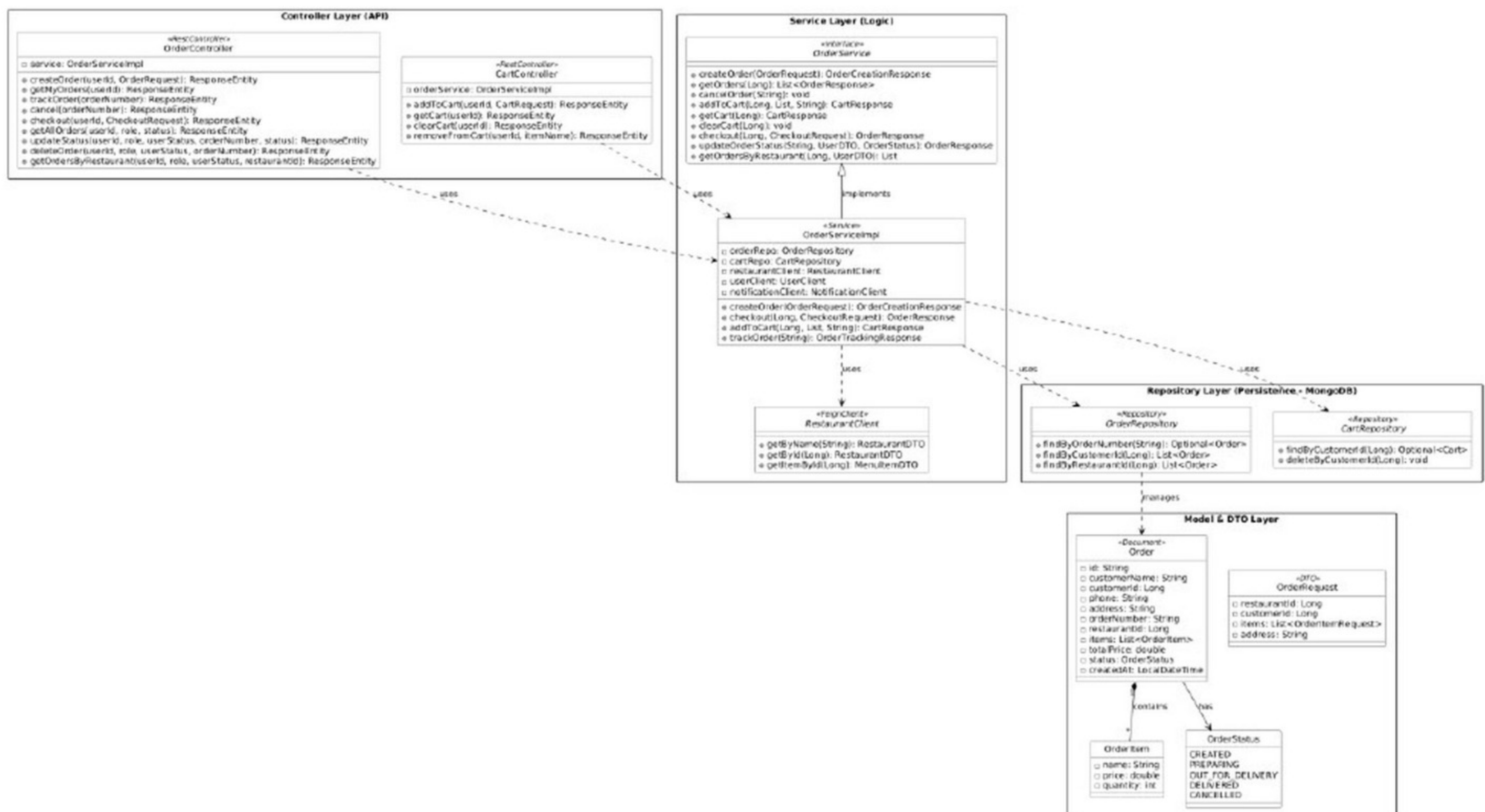
NoActiveDelivery: Delivery.allInstances()->forAll(d |
d.deliveryPersonId = request.deliveryPersonId implies
d.status = DeliveryStatus::DELIVERED or d.status =
DeliveryStatus::CANCELLED)

5.9 Class Diagram (Notification Service)

The provided Class Diagram illustrates the internal architecture of the Notification Service for the Online Food Ordering platform. It follows a structured Controller-Service-Repository pattern where the OrderController in the Order Service interacts with the NotificationClient to trigger alerts.

The core of the data model is the Notification entity, which includes essential attributes like id, orderNumber, message, and isRead status. The service is designed to be asynchronous, ensuring that system performance is not impacted by messaging delays. Furthermore, the system uses status checks to ensure notifications are

only triggered for valid order transitions, such as moving from **PREPARING** to **DELIVERED**.



OCL For Notification Service class Diagram:

1- context Notification inv MessageNotEmpty:

self.message <> null and self.message.size() > 0

2- context Notification inv RecipientValid: self.recipientId > 0

3- context

OrderServiceImpl::updateOrderStatus(orderNumber: String, dto: UserDTO, status: OrderStatus):

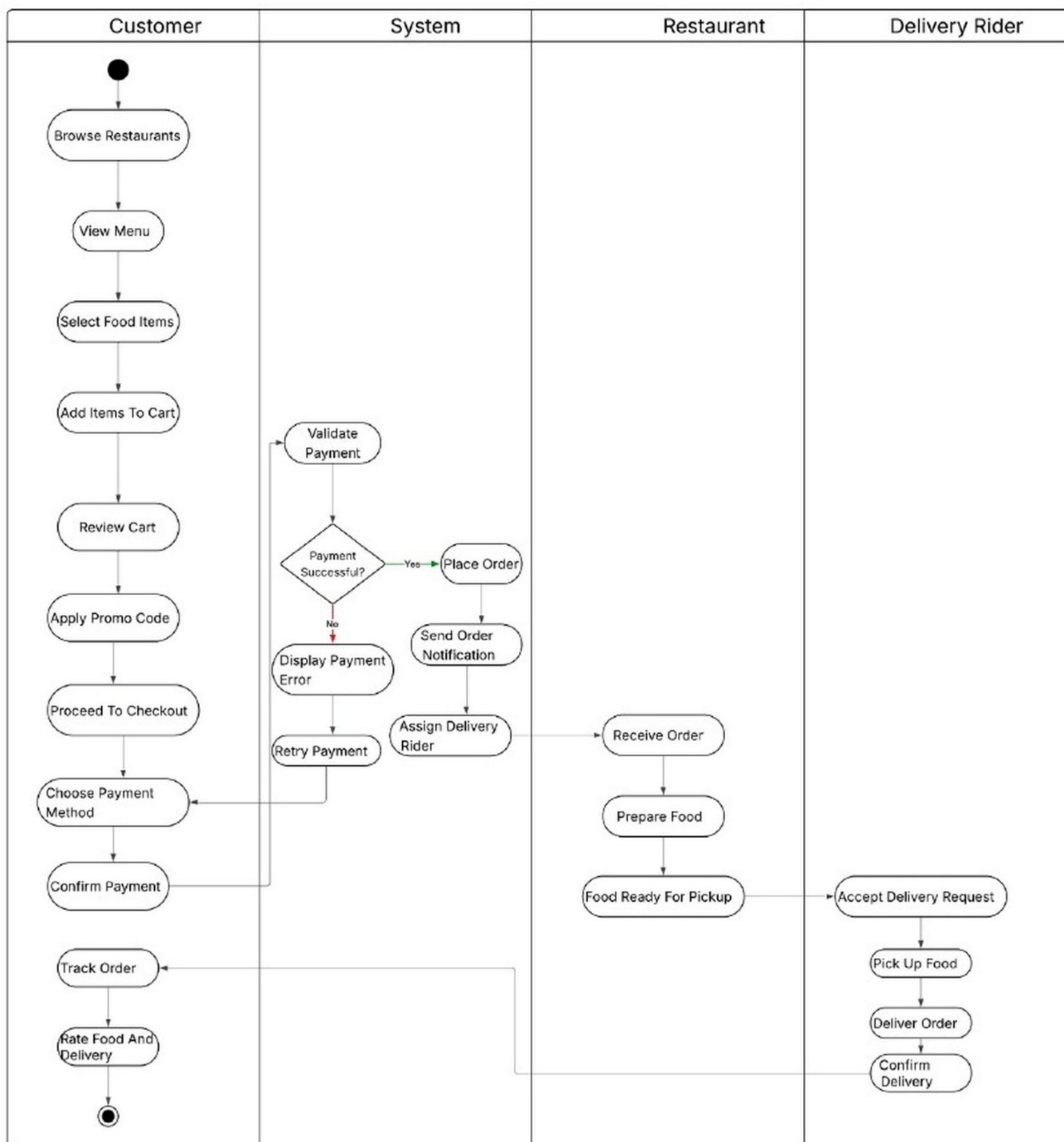
OrderResponse post NotificationTriggered: result <> null and result.status = status

5.10 Activity Diagrams

The provided Activity Diagrams illustrate the step-by-step operational workflows within the Online Food Ordering System, capturing how different actors interact with the system to complete specific business processes.

. Food Ordering Process: This diagram maps the primary customer journey, starting from browsing restaurants and viewing menus to selecting food items and managing the cart. It highlights the critical decision point of payment validation; if successful, the order is placed and notifications are dispatched.

Food Ordering Activity Diagram



.Authentication (Login & Register): These diagrams detail the security handshake between the user and the system. The system validates input data against the database, generates JWT tokens upon success, and creates active user sessions.

Figure 2 – Food Ordering Activity Diagram

Login Activity Diagram

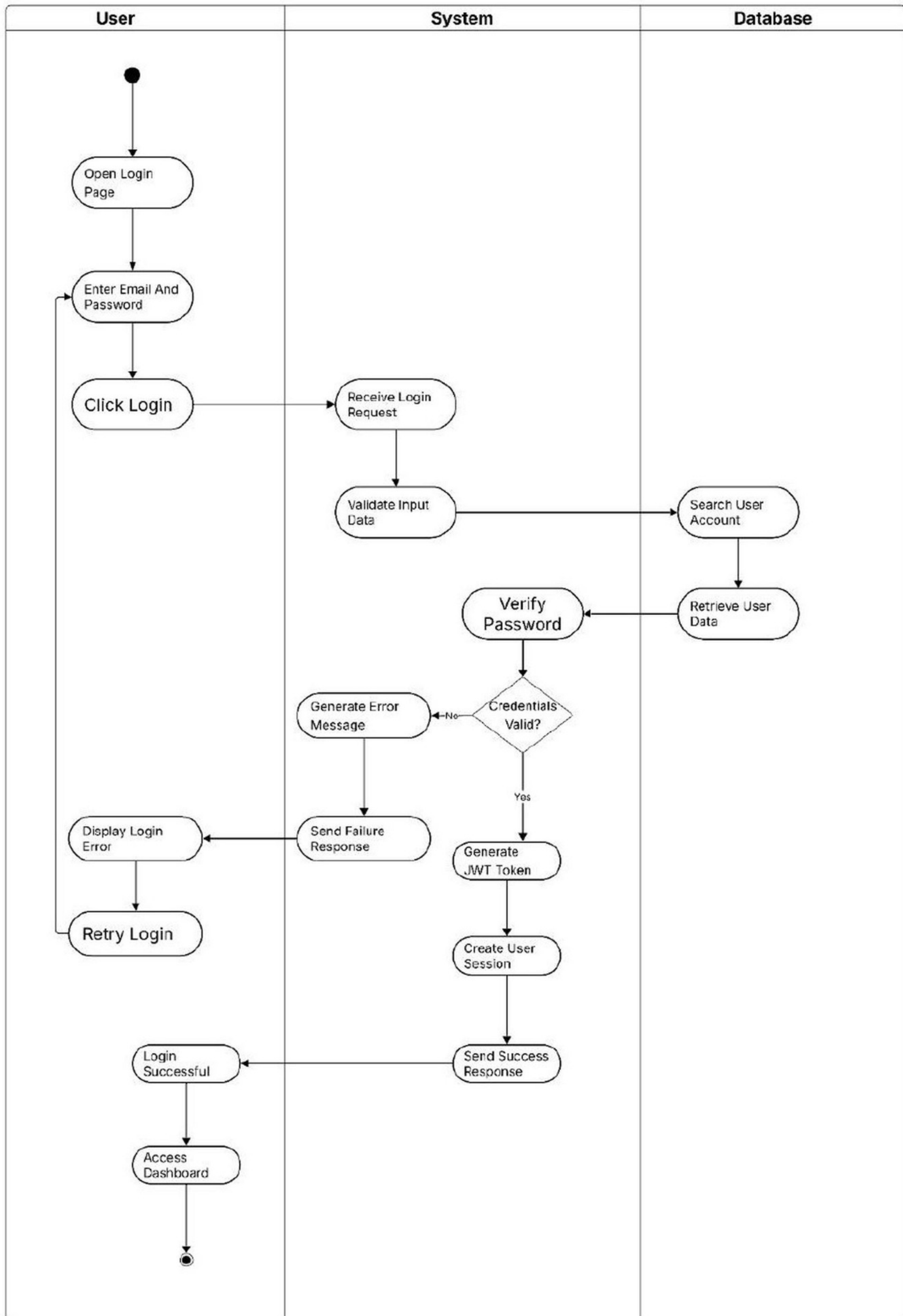


Figure 3 – Login Activity Diagram

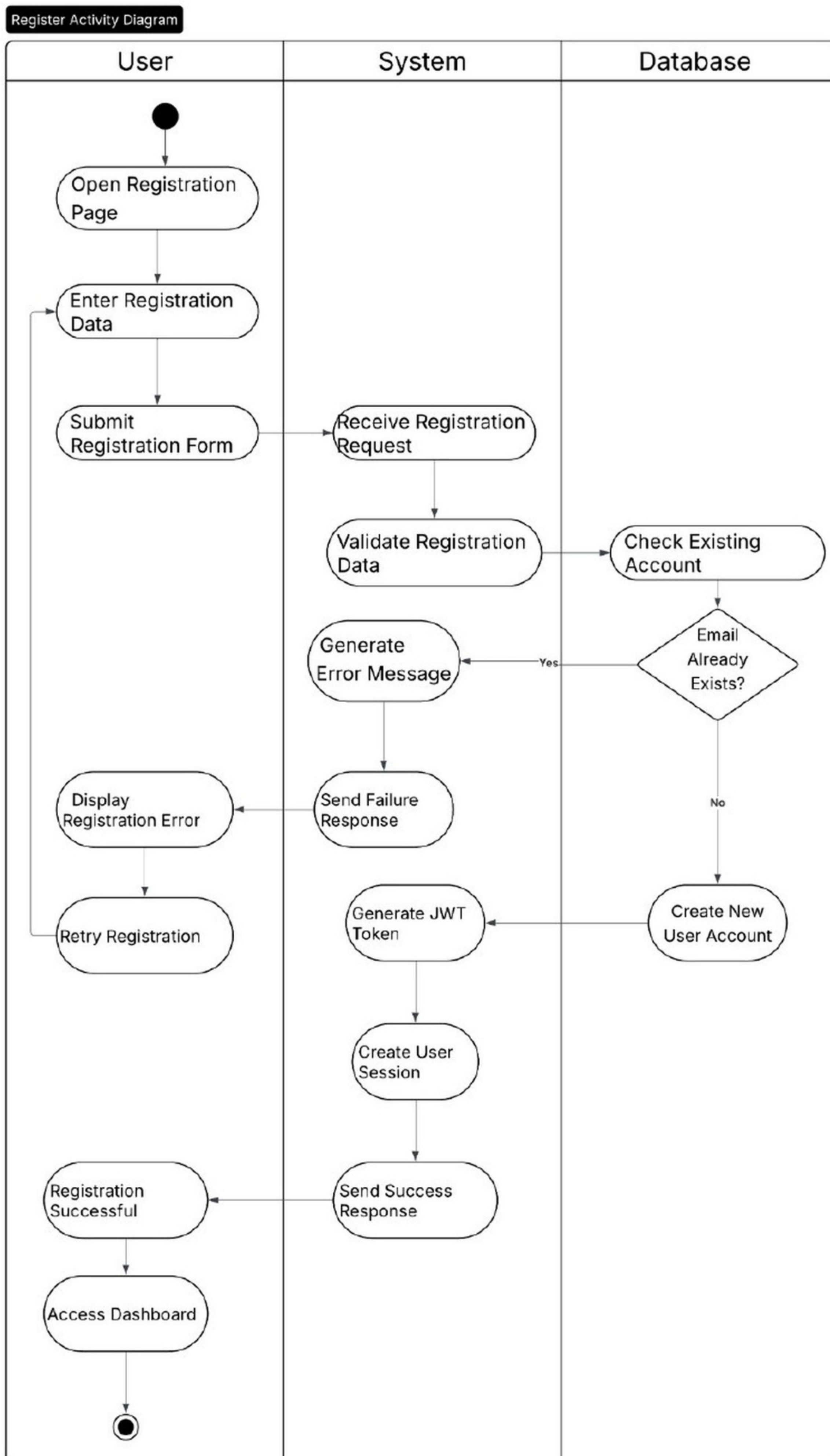


Figure 4 – Register Activity Diagram

.Restaurant Order Management: This workflow defines the responsibilities of the Restaurant Admin, from receiving a new order to marking it as "Food Ready For Pickup". It ensures the system appropriately notifies the delivery rider once preparation is complete.

Restaurant Order Management

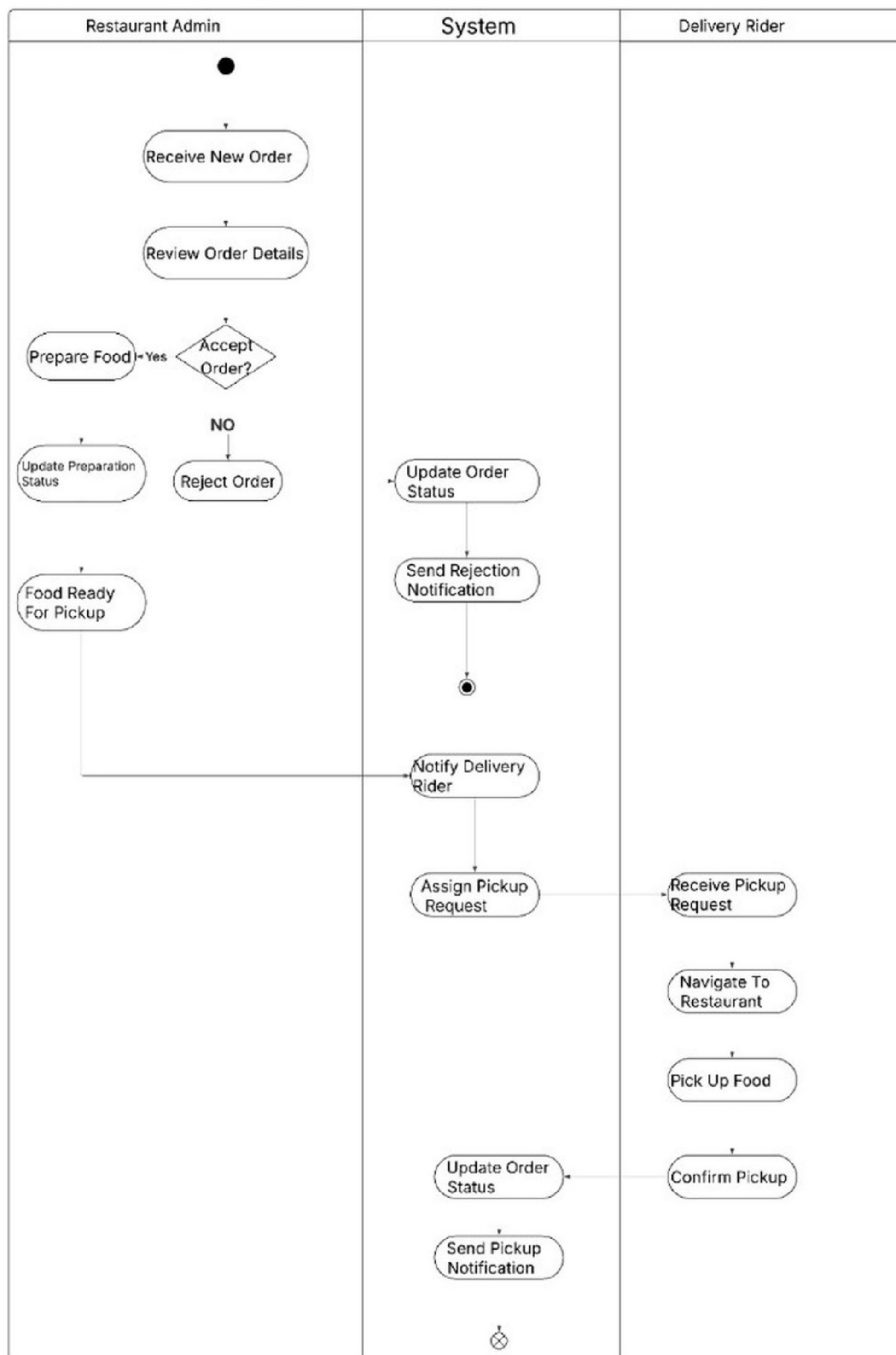


Figure 5 – Restaurant Order Management

.Delivery Process: This diagram coordinates the Delivery Rider and the Customer, showing the sequence from assigning a rider to the final confirmation of delivery. It includes logic for navigating to locations and the final customer action of rating the delivery service.

Delivery Process Activity Diagram

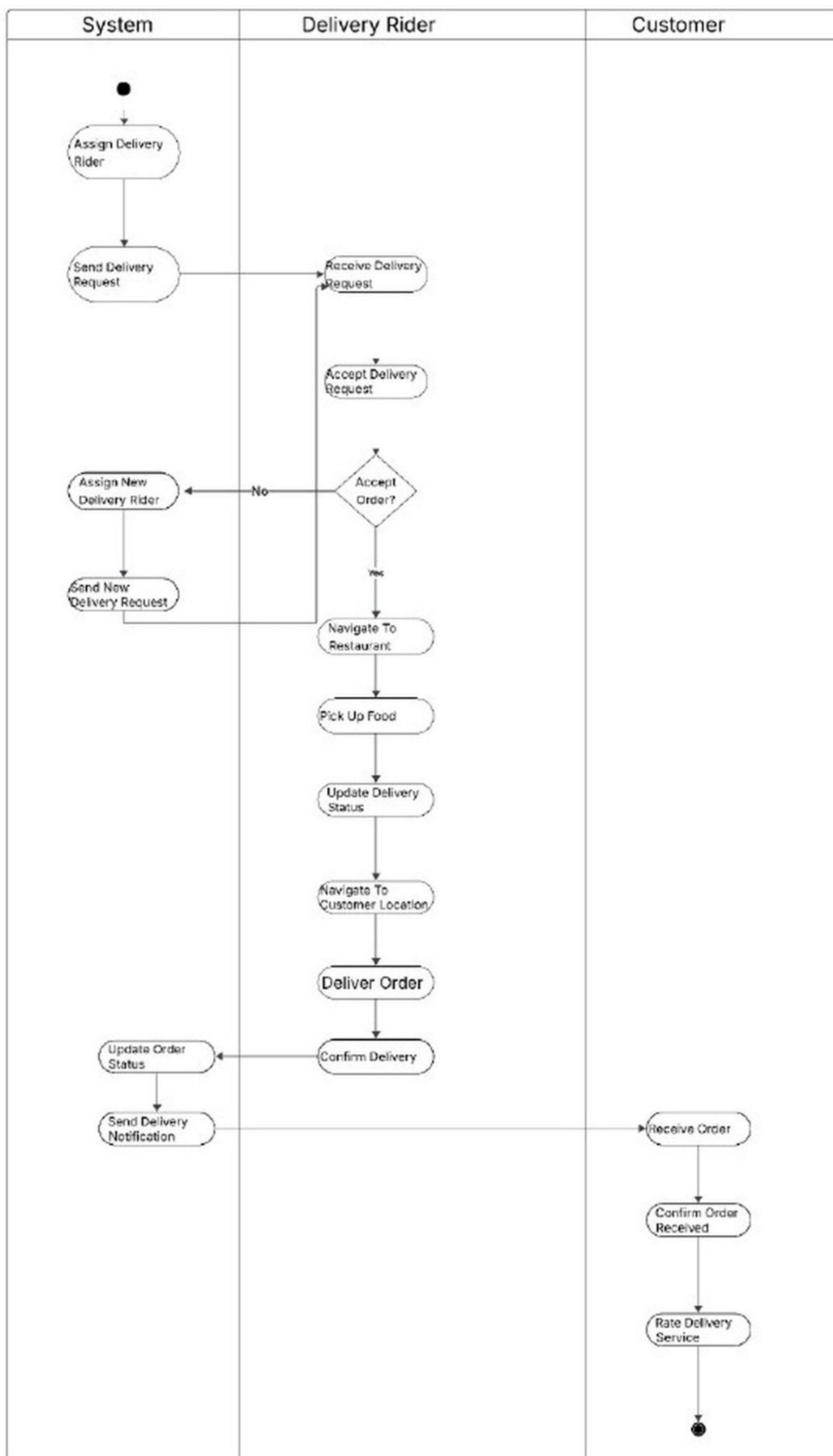
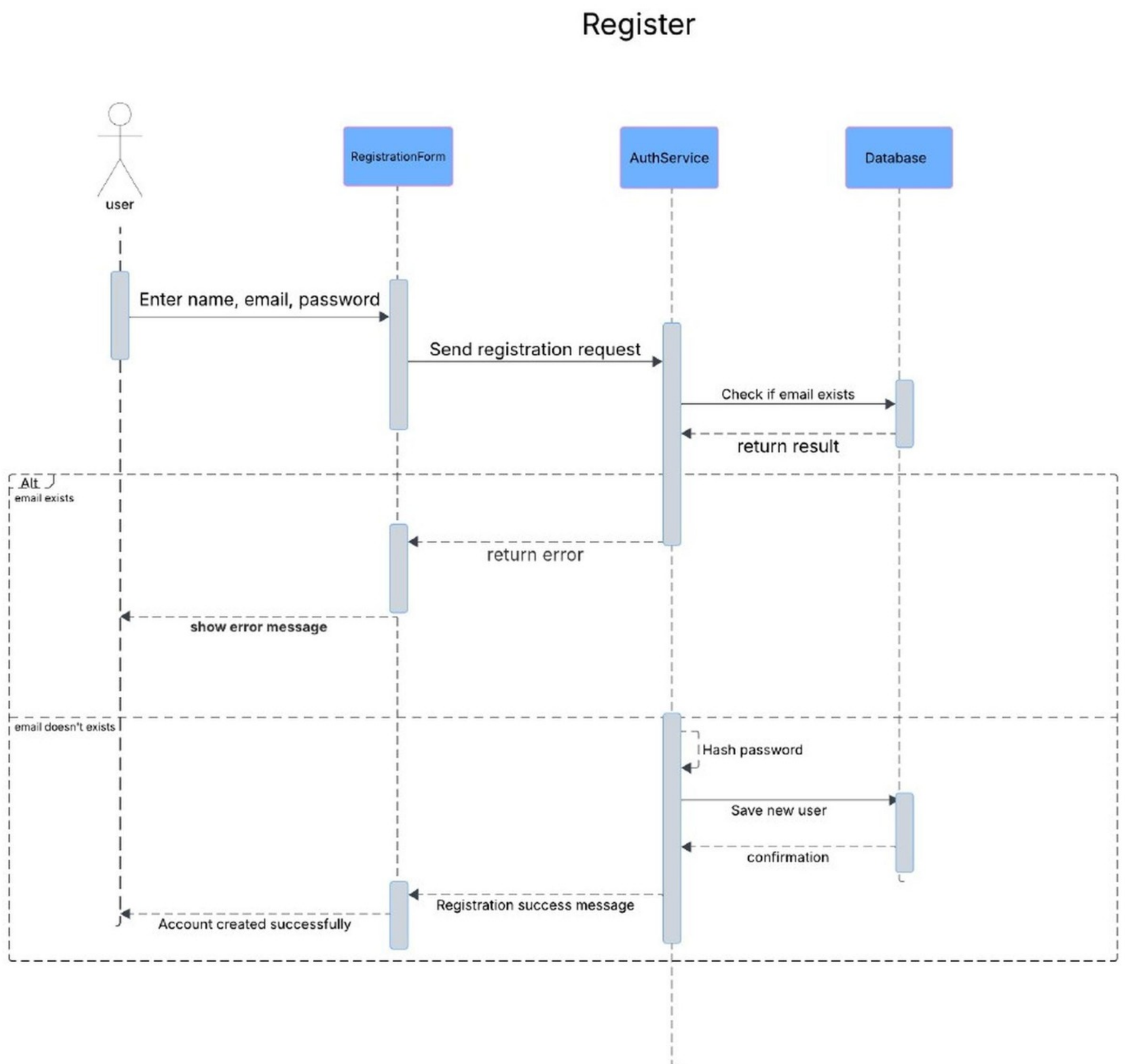


Figure 6 – Delivery Process Activity

5.11 Sequence Diagrams

The following Sequence Diagrams illustrate the chronological flow of messages and interactions between system components during key operational events.

. Registration Sequence: This diagram tracks the flow of user data from the registration form to the Auth Service. It utilizes an "Alt" fragment to handle conditional logic: if the database confirms the email exists, an error is returned; otherwise, the system hashes the password, saves the new user, and confirms successful account creation.



. Login Sequence: Highlights the stateless authentication process where the **Auth Service** verifies credentials against the database. Upon successful verification, it generates a JWT token, which is returned to the UI to grant the user access.

Figure 7 – Registration Sequence Diagram

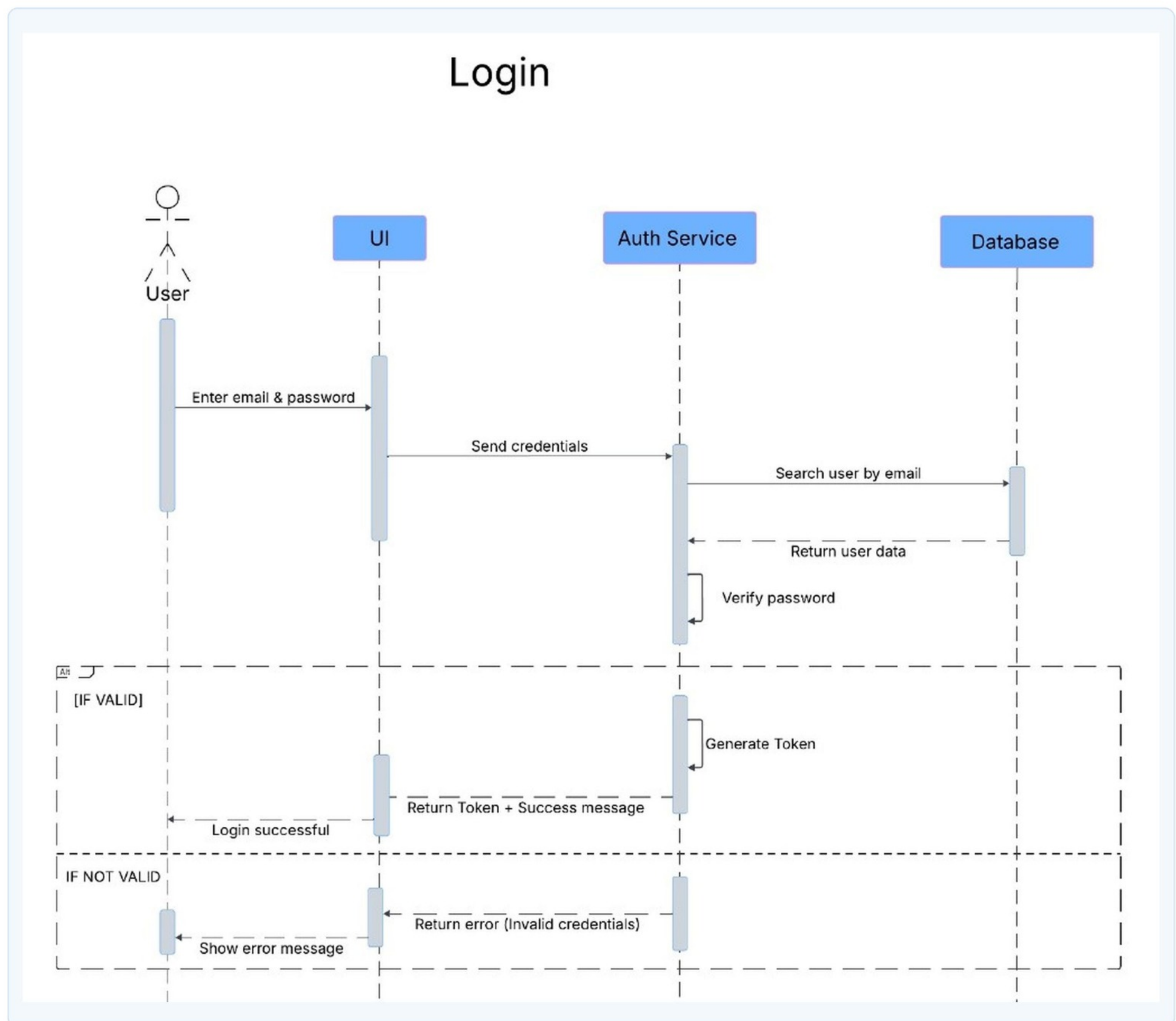


Figure 8 – Login Sequence Diagram

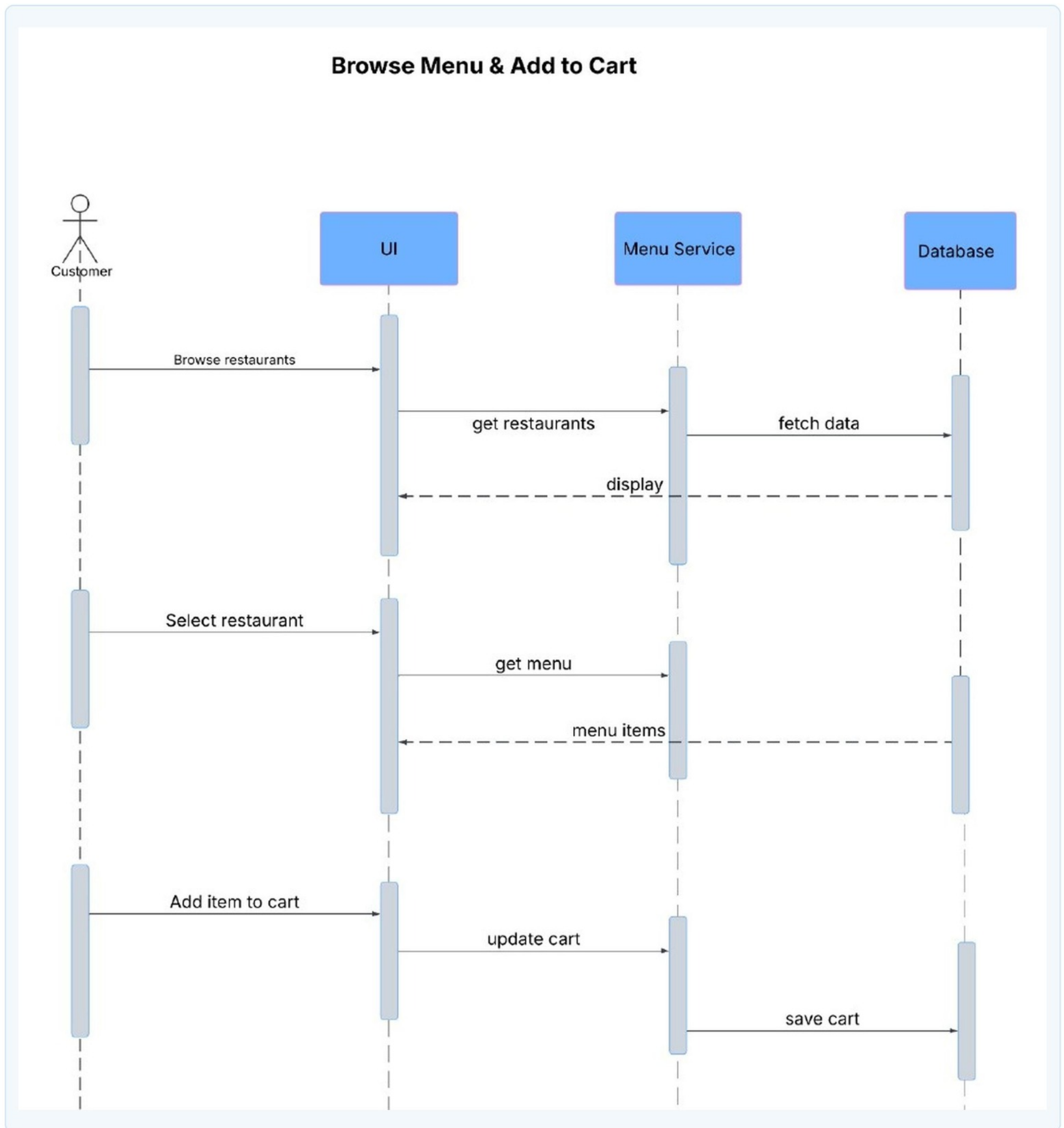


Figure 9 – Browse Menu & Add to Cart

request to the **Payment Gateway** and the subsequent success/failure branches that trigger the **Restaurant Service** to notify the admin of a new order.

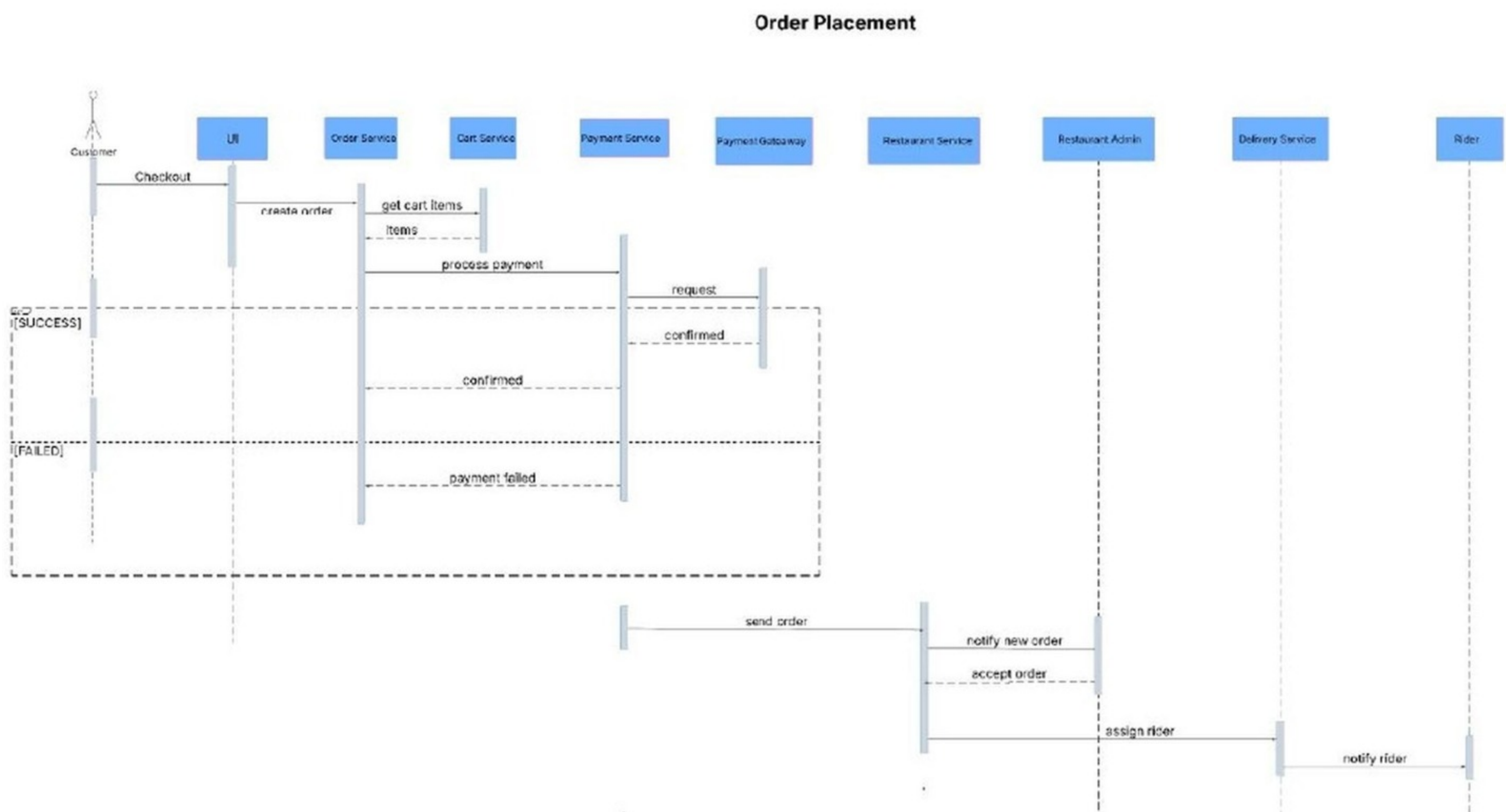


Figure 10 – Order Placement Sequence

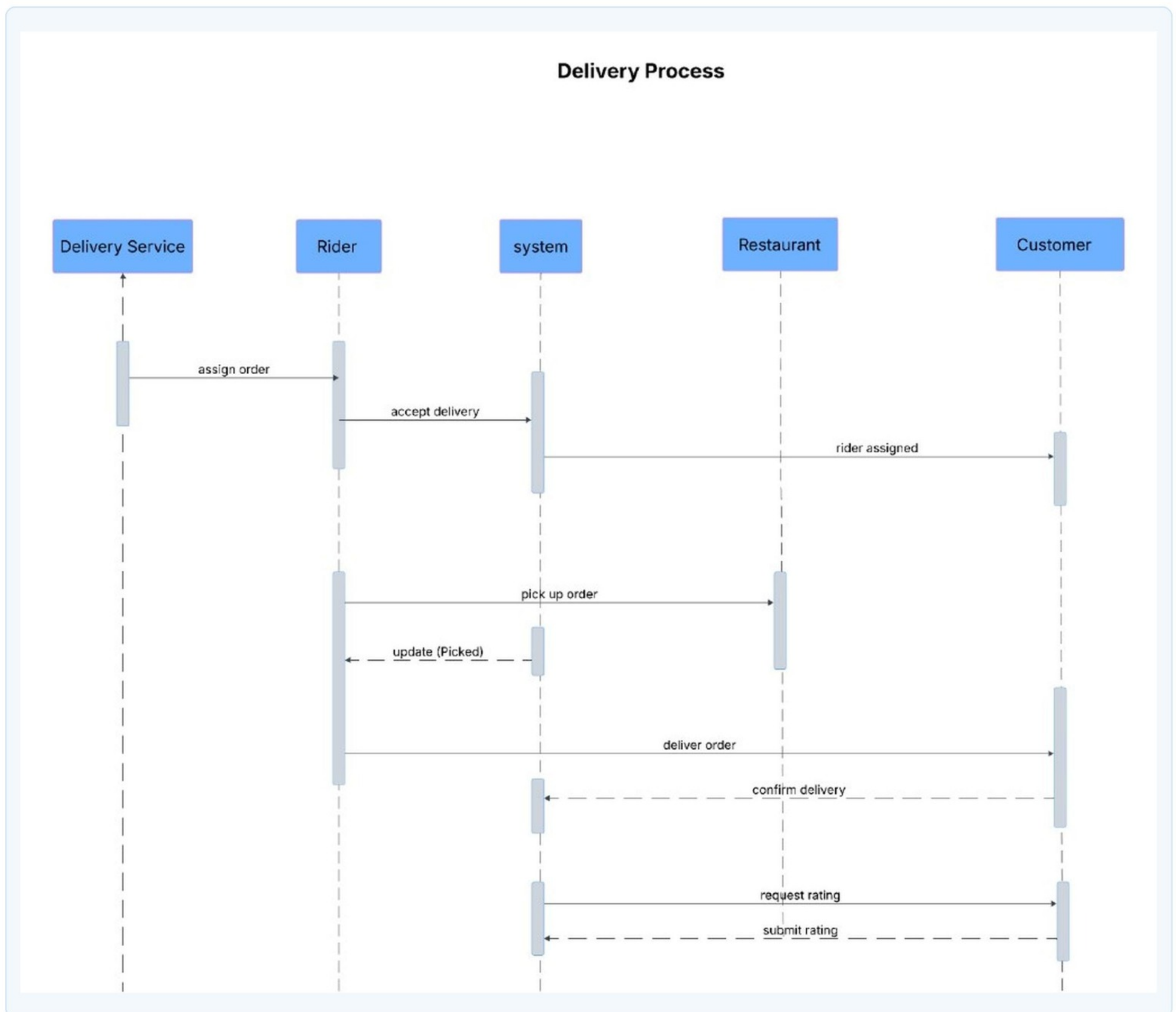


Figure 11 – Delivery Process Sequence

Team Information

Name	ID
Abdelrahman Mohamed Hamza	20230319
Abdelrahman Ibrahim Abdeltwab	20230306
Ezz El-deen Atef Mohamed	20230345
Farah Hassan Tawfiq	20230406
Mohamed Hosny Mohamed	20230465
Verena Thabet youssef	20230408

Thank You